

Music Scheduler - Database Backup & Restore System

This guide covers the complete database backup and restore system with GitHub integration for the Music Scheduler application.



Features

- **Manual Database Backups:** Create on-demand MySQL database backups
- **Automated Daily Backups:** Schedule automatic daily backups at 2:00 AM
- **GitHub Integration:** Automatically push backups to GitHub repository
- **Compression:** All backups are compressed with gzip to save space
- **Web Interface:** Admin dashboard for backup/restore operations
- **Backup Validation:** Automatic validation of backup files before restore
- **Pre-restore Safety:** Automatic backup creation before restore operations
- **Cleanup:** Automatic removal of backups older than 30 days
- **Audit Logging:** Complete logging of all backup and restore operations



Directory Structure

```
music-scheduler/  
├── scripts/  
│   ├── backup_db.sh           # Database backup script  
│   ├── restore_db.sh          # Database restore script  
│   ├── list_backups.sh        # List available backups  
│   └── setup_cron_backup.sh    # Setup automatic backups  
├── database_backups/          # Local backup storage  
│   ├── backup_log.txt         # Backup operation log  
│   ├── restore_log.txt        # Restore operation log  
│   └── *.sql.gz               # Compressed backup files  
├── uploaded_backups/          # Uploaded backup files for restore  
├── templates/  
└── admin_backup_restore.html  # Admin web interface
```



Setup Instructions

Prerequisites

1. **MySQL/MariaDB:** Database server with mysqldump and mysql client
2. **Git:** For version control and GitHub integration
3. **Web Server:** Flask application running
4. **Admin Access:** Admin user account in the music scheduler system

1. Database Backup System Setup

The backup scripts are already installed in the `scripts/` directory. Make sure they are executable:

```
chmod +x scripts/*.sh
```

2. GitHub Integration Setup

To enable automatic GitHub backups, configure git authentication:

Option A: Personal Access Token (Recommended)

```
# Configure git with your GitHub credentials
git config user.name "Your Name"
git config user.email "your-email@example.com"

# Store GitHub credentials (you'll be prompted for username and token)
git config credential.helper store
git push # Enter username and Personal Access Token when prompted
```

Option B: SSH Key Authentication

```
# Generate SSH key (if you don't have one)
ssh-keygen -t ed25519 -C "your-email@example.com"

# Add SSH key to GitHub account
cat ~/.ssh/id_ed25519.pub # Copy this to GitHub Settings > SSH Keys

# Update git remote to use SSH
git remote set-url origin git@github.com:dfultonthebar/music-scheduler.git
```

3. Automatic Backup Setup

Set up daily automatic backups at 2:00 AM:

```
# Open crontab editor
crontab -e

# Add this line for daily backups at 2:00 AM
0 2 * * * /full/path/to/music-scheduler/scripts/backup_db.sh >> /tmp/music_scheduler_backup_cron.log 2>&1
```

Alternative: Using the setup script (if crontab is available)

```
./scripts/setup_cron_backup.sh
```

4. Directory Permissions

Ensure proper permissions for backup directories:

```
mkdir -p database_backups uploaded_backups
chmod 755 database_backups uploaded_backups
chown -R $USER:$USER database_backups uploaded_backups
```

Usage

Web Interface (Recommended)

1. **Access Admin Panel:** Navigate to `/admin/backup-restore` in your browser
2. **Login:** Use admin credentials to access the interface
3. **Create Backup:** Click “Create New Backup” button
4. **View Backups:** Browse available backups in the list
5. **Download Backups:** Click download button next to any backup
6. **Restore Database:** Upload a backup file and click restore

Command Line Interface

Create Manual Backup

```
cd music-scheduler
./scripts/backup_db.sh
```

List Available Backups

```
./scripts/list_backups.sh
```

Restore from Backup

```
./scripts/restore_db.sh database_backups/backup_file.sql.gz
```

Configuration

Database Configuration

The scripts use environment variables from `.env` file:

```
# Database Configuration
DB_HOST=localhost
DB_USER=music_user
DB_PASSWORD=music_pass
DB_NAME=music_scheduler
```

Backup Retention Policy

- **Local Backups:** Kept for 30 days (configurable in `backup_db.sh`)
- **GitHub Backups:** Kept indefinitely (manually manage via GitHub)

Security Considerations

1. **File Permissions:** Backup files contain sensitive data - ensure proper permissions
2. **Database Credentials:** Store database passwords securely
3. **GitHub Access:** Use Personal Access Tokens with minimal required permissions
4. **Backup Validation:** Always validate backup files before restoration
5. **Pre-restore Backups:** Automatic pre-restore backups prevent data loss



Monitoring and Logs

Log Files

- `database_backups/backup_log.txt` : All backup operations
- `database_backups/restore_log.txt` : All restore operations
- `/tmp/music_scheduler_backup_cron.log` : Cron job execution log

Monitoring Commands

```
# View recent backup activity
tail -f database_backups/backup_log.txt

# View recent restore activity
tail -f database_backups/restore_log.txt

# Check cron job logs
tail -f /tmp/music_scheduler_backup_cron.log

# List all backup files with details
ls -lah database_backups/*.sql.gz
```



Troubleshooting

Common Issues

1. “mysqldump: command not found”

```
# Install MySQL client tools
sudo apt-get update
sudo apt-get install mysql-client-core-8.0
```

2. “Access denied for user”

```
# Check database credentials in .env file
# Verify user has LOCK TABLES and SELECT privileges
GRANT SELECT, LOCK TABLES ON music_scheduler.* TO 'music_user'@'localhost';
```

3. “Git push failed - authentication required”

```
# Set up GitHub authentication (see Setup Instructions above)
git config credential.helper store
```

4. “Permission denied” for backup files

```
# Fix file permissions
chmod +x scripts/*.sh
chmod 755 database_backups uploaded_backups
```

5. Cron job not running

```
# Check if cron service is running
sudo systemctl status cron

# Check cron logs
grep CRON /var/log/syslog

# Verify cron job is installed
crontab -l
```

Testing the System

Test Manual Backup

```
cd music-scheduler
./scripts/backup_db.sh
ls -la database_backups/
```

Test Backup Restore

```
# Create a test backup
./scripts/backup_db.sh

# Test restore (will prompt for confirmation)
./scripts/restore_db.sh database_backups/music_scheduler_backup_YYYYMMDD_HHMMSS.sql.gz
```

Test Web Interface

1. Start the Flask application
2. Login as admin user
3. Navigate to `/admin/backup-restore`
4. Test creating and downloading backups

Maintenance

Regular Tasks

1. **Monitor Disk Space:** Check backup directory size regularly
2. **Verify Backups:** Periodically test backup restoration
3. **Update Credentials:** Rotate GitHub tokens and database passwords
4. **Review Logs:** Check backup/restore logs for errors
5. **Clean Old Backups:** Remove old backups if needed (done automatically)

Backup File Naming Convention

```
music_scheduler_backup_YYYYMMDD_HHMMSS.sql.gz
```

Example: `music_scheduler_backup_20250813_033523.sql.gz`

Recovery Procedures

Full System Recovery

1. **Restore Application Code:**

```
bash
git clone https://github.com/dfultonthebar/music-scheduler.git
cd music-scheduler
```

2. **Restore Database:**

```
bash
./scripts/restore_db.sh database_backups/latest_backup.sql.gz
```

3. **Restart Services:**

```
bash
# Restart Flask application
# Restart web server
# Verify functionality
```

Point-in-Time Recovery

1. Choose appropriate backup file from the list
2. Use restore script with confirmation
3. Verify data integrity after restoration

Support

For issues or questions:

1. Check the troubleshooting section above
2. Review log files for error details
3. Consult the application documentation
4. Contact system administrator

Last Updated: August 13, 2025

Version: 1.0

Author: Music Scheduler Development Team